

VentureFlow

AI due diligence for pitch decks

How it works, what was trained, and what it can and cannot do

What this document is

VentureFlow reads a startup's pitch deck and checks it. It pulls out the claims the deck makes, searches the web to see whether they hold up, looks up who founded the company, scores the business against a model trained on real startup outcomes, and writes an investment memo.

This document explains the whole thing from scratch - what problem it solves, how the pipeline works end to end, which machine-learning models were trained and exactly how, which public datasets they learned from, and where the system is weak. It is written to be readable without any prior knowledge of the codebase.

The one idea behind every design decision

A tool that quietly invents a number is worse than no tool at all, because a convincing fabrication is impossible to tell apart from a real finding. Most of the engineering here went into making the system's own failures visible rather than into adding features. That principle explains nearly every choice in the pages that follow.

1. The problem, in plain terms

An investor receives far more pitch decks than anyone can study properly. Reading a deck is quick. **Checking** it is the expensive part.

A deck asserts things: a market worth billions, a growth rate, a list of customers, a founding team with the right background. Verifying even one of those means searching, reading sources and forming a judgement. Multiply that by fifty decks a week and the checking simply does not happen - which is precisely when a weak company slips through on a confident-sounding slide.

VentureFlow automates that checking pass. Give it a deck and it will:

- Pull out every specific claim the deck makes.
- Search the public web and judge each claim as supported, refuted or unverifiable - with the sources it used attached.
- Find the founders (even when the deck never names them) and check their background against public record.
- Score the company against a model trained on 1,298 real startups whose outcomes are known.
- Write a structured memo, and tell you how much of the deck it actually managed to read.

It is a triage tool, not a decision maker

The score separates more-promising from less-promising decks well enough to sort a pile. It is nowhere near good enough to decide an investment, and Section 7 gives the exact numbers rather than a reassuring adjective.

2. The technology

Nothing exotic. The interesting parts are the trained models in Section 5 and the honesty checks in Section 4, not the framework choices.

Layer	Technology	What it does here
Backend	Python 3.13, FastAPI, uvicorn	The API the frontend talks to; runs the analysis pipeline.
Language model	Groq - openai/gpt-oss-120b	Reads decks, judges claims, writes the memo.
Database	Neon (serverless Postgres) + pgvector	Stores reports; pgvector powers similarity search.
Machine learning	LightGBM, scikit-learn, TF-IDF + SVD	The four shipped models.
Web search	DuckDuckGo (ddgs)	Evidence for claim checks and founder lookups.
Deck parsing	pdfplumber, python-pptx, python-docx	Turns a deck into text. No OCR.
Frontend	React + TypeScript, Vite, Tailwind, recharts	The dashboard and report UI.
Hosting	Vercel (frontend), Neon (data)	Deployment.

3. How the pipeline works

The diagram below is the whole system. Read it top to bottom: a deck comes in at the top and a memo comes out at the bottom. **Purple boxes are trained machine-learning models.** **Green boxes are the two honesty checks** that Section 4 explains.

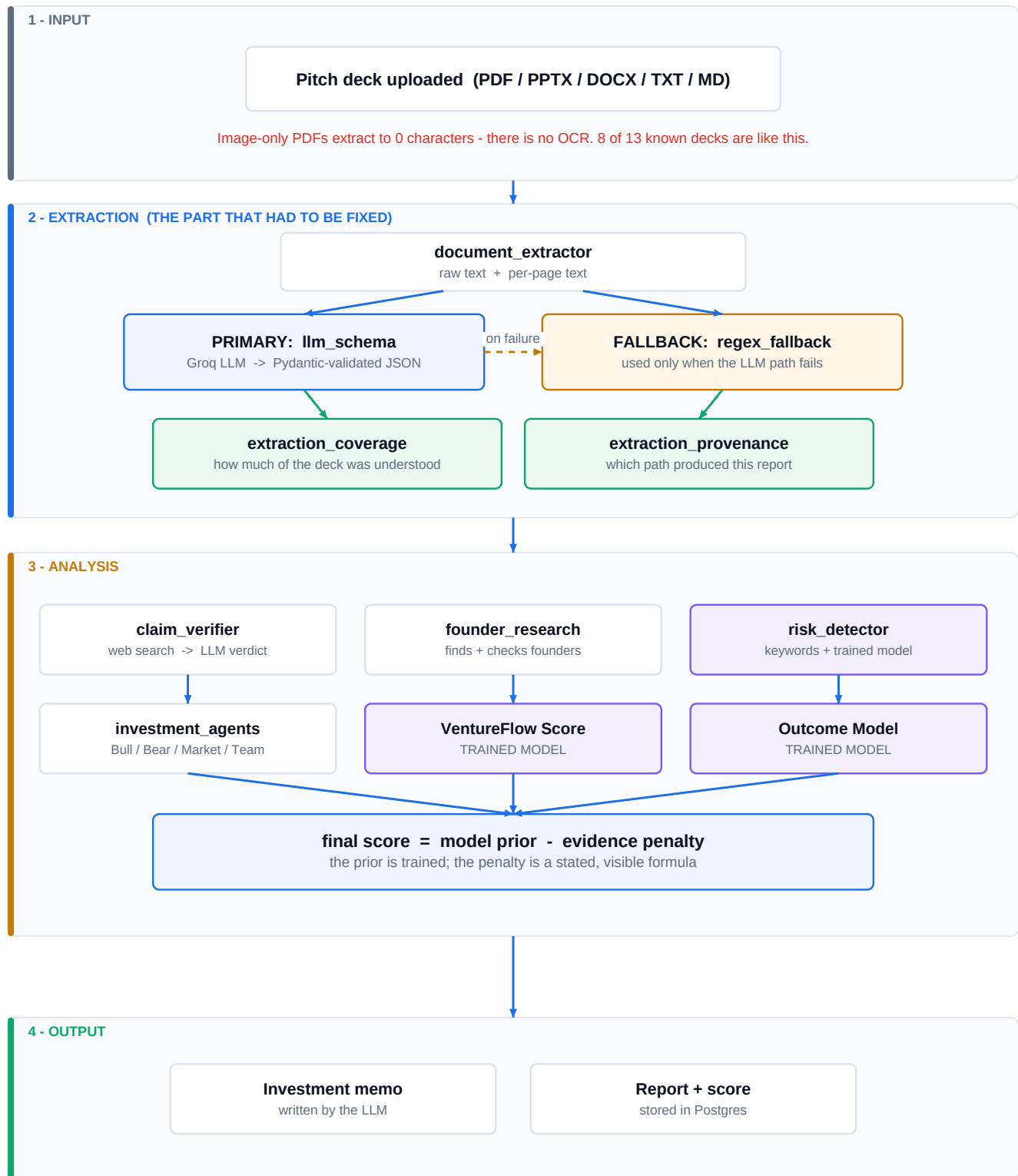


Figure 1 - The VentureFlow analysis pipeline, end to end.

3.1 Why extraction has two paths

Look again at band 2 of the diagram. Extraction - turning a deck into structured facts - has a primary path and a fallback, and everything downstream depends on which one ran.

The **primary path** asks the language model for a fixed JSON structure and validates it with Pydantic before anything is allowed to trust it. It understands a deck by its content, so a slide titled "1-Click Car Service" is correctly recognised as the solution slide.

The **fallback path** uses hand-written pattern matching. It is much worse: on the real 2008 UberCab deck it finds 3 claims where the primary path finds 14. It exists so that an outage degrades the product instead of breaking it.

The bug that shaped this entire project

One line of code called a variable named `prompt` that did not exist. Every call raised an error; a catch-all handler swallowed it; extraction silently dropped to the fallback path. **Every analysis for an unknown number of weeks was produced by the worse path, and nothing anywhere said so.** It was found only when a human compared a report against the original PDF by hand. The lesson was not "fix the typo" - it was that a failure nobody can see will run for weeks.

4. The two honesty checks

These are the green boxes in the diagram, and they are what makes the rest trustworthy.

4.1 Extraction coverage - did we actually read the deck?

Before this existed, two completely different situations produced identical output. If the report said "insufficient data", that could mean the deck genuinely said very little - or it could mean the deck was full of information and the parser dropped it. The first is an honest finding. The second is a bug. A reader could not tell which they were looking at.

Coverage measures one thing: **of the text we were given, how much can be found again in the structured output?** The headline figure is the share of slides that contributed at least one line, because the failure being guarded against is whole slides vanishing. The report names the slides that contributed nothing, so you can check it against the PDF in about ten seconds.

Measured on the real deck corpus	Before the fix	After the fix
Average coverage across decks	20.8%	69.6%
Total claims extracted	14	92
Uber 2008 deck - claims found	3	14

4.2 Extraction provenance - which path produced this?

Every report now records whether it came from the good path or the fallback. When the fallback runs, it says so in four places at once: the logs, the stored report, the written memo, and a red banner in the interface. A test deliberately breaks the language-model path and fails if that degradation is not visible.

4.3 The same principle, applied everywhere else

- **Founder search** reports three separate states: *the search failed*, *the search was not attempted*, and *we searched and genuinely found nobody*. Only the last one says anything about the company. They are three different colours in the interface.
- **Founder names** must be grammatically credited as a founder in a real source, not merely appear on the same page - otherwise a directory listing turns a random executive into a fabricated co-founder.
- **Missing numbers** display as a dash, never as zero. A figure the deck never stated and a figure that is genuinely zero are different facts.
- **A corpus deck** that turned out to be a business-school case study rather than the company's own fundraising deck is flagged as unverified rather than quietly used as ground truth.

5. The machine-learning models

Six models were trained. **Four are used in the product. Two were deliberately left out**, and the reasons they were left out are as informative as the ones that shipped.

First, a plain-language note on how to read the numbers in this section, because one term does most of the work.

What "AUC" means

AUC answers: given one company that succeeded and one that failed, how often does the model give the successful one a higher score? **0.5 is a coin flip - completely worthless. 1.0 is perfect.** Real-world startup prediction is genuinely hard, so anything meaningfully above 0.5 is a real signal, and anything near 0.9 in this domain should make you suspect a leak rather than a breakthrough. That suspicion turned out to be correct twice here.

5.1 VentureFlow Score - the headline number

Property	Value
File	ml/models/venturescore_model.pkl
Method	Ensemble of 12 Random Forests, with isotonic calibration
Training data	1,298 Y Combinator companies
Features	22 numeric + 5 categorical, expanded to 80 columns
Cross-validated AUC	0.6772 (95% confidence interval 0.6607 - 0.6932)
Calibration error	0.0219 (lower is better; this is good)
Verified out-of-sample AUC	0.6356 on 261 companies with zero overlap with training

What it predicts. Not returns - *survival*. The label is whether a company was acquired, went public, or is still operating, versus having shut down. Companies younger than three and a half years are excluded entirely, because "still alive" tells you nothing about a company founded last year.

How it was trained. Each company becomes a row of features - its industry, its funding stage, tags describing what it does, how long its description is, and so on. A Random Forest learns which combinations preceded survival. Twelve of them vote, and isotonic calibration then adjusts the output so that "70% likely" genuinely

means roughly seven in ten, rather than being an arbitrary confidence number.

Three features were deliberately thrown away

Team size, company age and batch year were all removed. Each is recorded *years after* the outcome being predicted: team size measures growth that already happened, and age encodes whether a company had time to fail. Keeping them would have produced a better-looking score that had partly learned the answer instead of predicting it.

For comparison, the hand-written formula this replaced scores an AUC of exactly 0.5 - not approximately, exactly. It read no company feature at all, so every company received the same score and every comparison was a tie.

5.2 Outcome Model - a second opinion

A different algorithm on the same underlying data, kept as a cross-check. It uses LightGBM (gradient-boosted trees) over both the company's text description and its structured fields. Trained on 1,560 companies: 1,248 for training, 312 held back for testing.

All five variants that were trained are recorded, not just the flattering one:

Variant	AUC	Accuracy	Shipped?
Text only	0.5721	55.5%	No
Structured only, with hindsight features	0.7047	65.1%	No - leaks
Combined, with hindsight features	0.7039	65.1%	No - leaks
Structured only, deployable	0.6613	62.5%	No
Combined, deployable	0.6459	63.8%	Yes

The best-scoring model was deliberately not shipped

The two 0.70 variants reach that number by using team size and company age - the same after-the-fact features excluded from the VentureFlow Score. Removing them **costs 0.058 AUC**, and that cost is written into the model's own report file rather than hidden. A 0.70 that partly knows the answer is worth less than a 0.65 that does not.

5.3 Risk Disclosure Model - the strongest result

This one detects genuine risk language, as opposed to the boilerplate every company copies into its filings. It is trained on real regulatory documents rather than on startup data.

Property	Value
Training data	941 excerpts drawn from 614 real SEC filings (10-K, 10-Q, 8-K)
Balance	464 genuine-risk / 477 boilerplate
Labelling method	Weak supervision - 38 labelling functions (Ratner et al., NeurIPS 2016)
Split	Grouped by filing ID, so no single document spans training and testing

Property	Value
Held-out accuracy	96.6% - AUC 0.9973
Independent benchmark	Precision 92.9%, recall 86.7%, F1 0.897

What weak supervision means. Nobody hand-labelled 941 documents. Instead, 38 search phrases were written, each one strongly associated with either genuine risk or boilerplate, and the phrase that retrieved a passage became its label. It is far cheaper than human annotation and noisier - and the provenance file states this in as many words: *"labels are the retrieval phrase's bucket, NOT human judgement."*

The caveat the model records about itself

Its ranking AUC is **1.00 on SEC filings** but **0.778 on pitch decks**. It is close to flawless at the thing it was trained on and noticeably weaker at the thing it is actually used for. That gap sits in the model's own report file rather than being left for someone to discover later.

5.4 Text embedder

A supporting model. It converts any piece of text into 64 numbers so that similar text ends up with similar numbers, which is what powers "find me companies like this one". Built with TF-IDF and SVD rather than a modern sentence transformer, because the machine it was built on could not reach the model hub. When it fails it returns nothing at all rather than a vector of zeros - a zero vector would silently produce confident, meaningless neighbours.

5.5 Claim Model - trained, measured, deliberately not used

The idea was to replace the language model's claim checking with something cheaper: a classifier deciding whether evidence supports, refutes, or says nothing about a claim. It was trained on SciFact, 1,109 claim-and-evidence pairs.

Class	Precision	Recall	F1	Support
Not enough info	0.735	0.602	0.662	83
Refutes	0.063	0.042	0.050	48
Supports	0.393	0.527	0.451	91
Overall accuracy			0.450	222

A useful failure

45% accuracy on a three-way choice is barely above guessing - and the class that matters most, **Refutes**, is essentially never detected: an F1 of 0.05. Catching a false claim is the single most valuable thing a diligence tool can do, and this model cannot do it. Deciding whether evidence contradicts a claim requires understanding negation and relationships, which word-overlap statistics simply cannot represent. **It is not wired into the product, because shipping it would make the product worse.** It stands as evidence for keeping the language-model approach.

5.6 Risk/Tone Model - trained, usable, still not connected

Trained on Financial PhraseBank and a Twitter financial-news sentiment set - 15,376 examples, 77% accuracy, macro-F1 0.62. Genuinely usable, unlike the claim model. It is not connected because replacing the existing keyword-based risk detector requires a proper head-to-head comparison, and doing that carelessly would mean swapping one component for another without knowing which is better.

6. The datasets

Every dataset is public, named and verifiable. Nothing was scraped from an unattributed source, and no training data was invented or synthesised.

Dataset	Direct link	Trains	Size
Y Combinator company directory	github.com/yc-oss/api	VentureFlow Score, Outcome Model, embedder	1,560 companies
SEC EDGAR filings	efits.sec.gov/LATEST/search-index sec.gov/Archives/edgar/data	Risk Disclosure Model	941 excerpts / 614 filings
SciFact (Allen Institute for AI)	github.com/allenai/scifact	Claim Model (not shipped)	1,109 pairs
Financial PhraseBank	huggingface.co/datasets/takala/financial_phrasebank	Risk/Tone Model (not shipped)	part of 15,376
Twitter Financial News Sentiment	huggingface.co/datasets/zeroshot/twitter-financial-news-sentiment	Risk/Tone Model (not shipped)	part of 15,376
Real pitch decks	Source URL + SHA-256 recorded per file in the repository	Evaluation only - never training	7 decks

6.1 Where the labels come from

- **Y Combinator data** is a public, continuously updated mirror of YC's own company directory. A company's status field gives the label: acquired, public or active counts as a success; dead counts as a failure.
- **SEC filings** are public-domain regulatory documents - the primary source, not a summary of one. Labels come from the 38 retrieval phrases described in Section 5.3.
- **SciFact** ships with expert annotations already; no labelling was needed.
- **The pitch decks** are used only to test the system, never to train it. Each one has its source URL and a cryptographic hash recorded so anyone can confirm the exact file used.

Two headline numbers were retracted

An earlier evaluation reported 0.9747 for the Outcome Model and 0.668 for the VentureFlow Score. Both were **measured on the models' own training data** - the equivalent of marking an exam with the answer sheet. They were re-measured properly, corrected to 0.6459 and 0.6356, and the retraction is recorded in the repository rather than quietly overwritten.

7. What it cannot do

Stated plainly, because each of these will otherwise look like a bug someone introduced.

Limitation	Detail
Image-only decks cannot be read at all	8 of 13 well-known decks tested extract zero characters - every page is a picture. There is no OCR. Confirmed for Dropbox, LinkedIn, YouTube, Facebook, WeWork, BuzzFeed, Brex and Alan.
The scores are weak predictors	AUC 0.64-0.68 is a real signal and far better than the 0.5 it replaced, but it is a way to sort a pile of decks, not a way to decide an investment.
It predicts survival, not returns	A company that quietly survived as a small business scores the same as one that returned a fund.
The comparison set is YC-only	Similar companies are drawn from Y Combinator alumni, not from the whole market.
Free-tier language model limits	200,000 tokens per day, which is roughly 4 to 8 full deck analyses. Past that, extraction drops to the fallback path - and now says so.
No user accounts	The deployed demo is protected by a shared passphrase. That is a gate, not authentication: there is no user model and no per-account separation.
One search provider	DuckDuckGo only, and it rate-limits aggressively under sustained use.

8. Running it yourself

Two terminals, from the project root:

```
.venv/Scripts/python.exe -m uvicorn api:app --reload --port 8000
npm run dev --prefix frontend
```

Then open **localhost:5173**. From a clean machine you also need to install the Python and Node dependencies, provide a database URL and a Groq API key in a `.env` file, and apply the database migrations. The test suite is 406 tests and takes about five minutes.

9. Where to read more

Document	Contents
ml/README.md	Every model, in full technical depth
ml/research/README.md	Research write-up, methodology and related work
ml/research/EXPERIMENT_LOG.md	What was tried, including everything that failed
HANDOFF.md	Backend architecture, environment variables, known issues
frontend/FRONTEND_HANDOFF.md	Frontend design system, accessibility, deployment
ml/eval/validation/	Out-of-sample validation and the locked evaluation splits

In one sentence

VentureFlow does the checking pass on a pitch deck that an investor rarely has time to do - and, unusually, it is built to tell you clearly when it could not.